

PART B: Temosho Mapheto Individual Practical

602009

Module

DevOps Toolchain Setup and Hands-On Exercises

Introduction

The objective of Part B was to demonstrate practical skills using Docker, Git, and Chef. The exercise involved creating and running a Docker container, managing source code with Git, and automating server configuration using Chef. Ubuntu Linux was used as the development environment throughout the practical.

1. Docker Practical Exercise

Objective

To build, tag, and run a custom Docker container that executes a simple Python application.

Step 1: Create a Project Directory

A project directory was created to store the application files.

```
mkdir docker-project
```

```
cd docker-project
```

Step 2: Create a Python Script

A Python file named app.py was created.

```
nano app.py
```

The following code was added:

```
print("Hello World")
```

Step 3: Create a Dockerfile

A Dockerfile was created to define the container image.

```
nano Dockerfile
```

Contents:

```
FROM python:3
```

COPY app.py /app.py

CMD ["python3", "/app.py"]

Explanation:

- FROM python:3 uses the official Python image.
- COPY transfers the application file into the container.
- CMD executes the Python script when the container starts.

Step 4: Build and Tag the Image

```
sudo docker build -t hello-world-app .
```

This command created a Docker image called hello-world-app.

Step 5: Run the Container

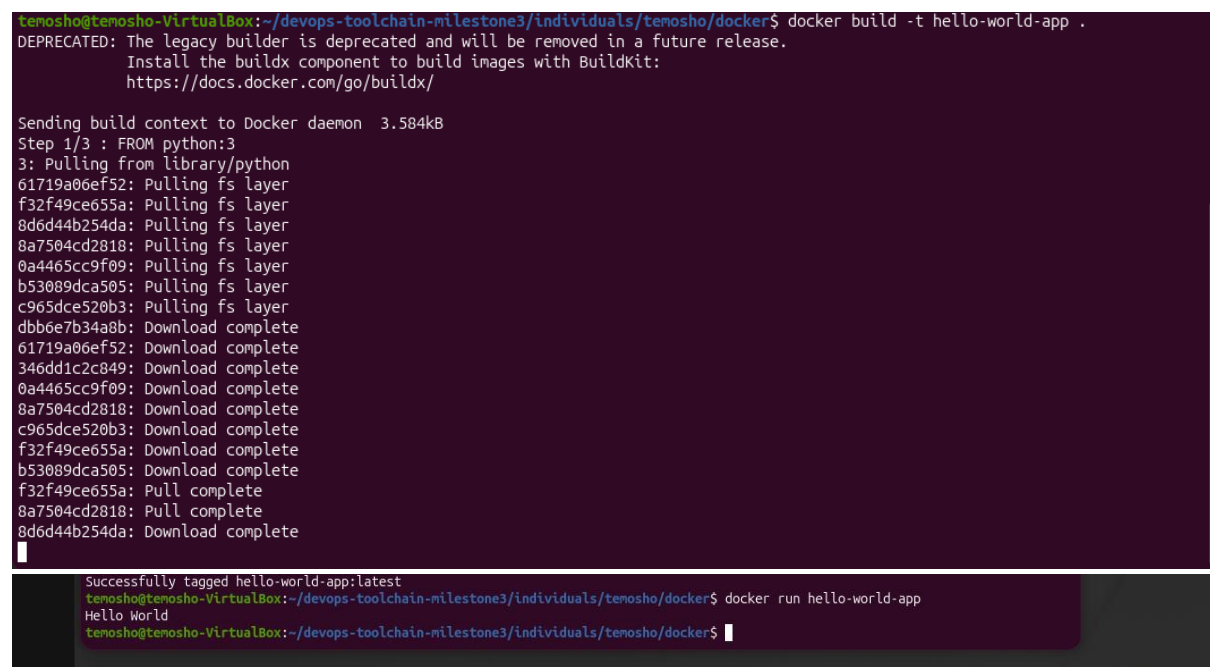
```
sudo docker run hello-world-app
```

Output:

Hello World

This confirmed that the Docker container executed successfully.

Screenshots



The first screenshot shows the execution of the command `docker build -t hello-world-app .` in a terminal. It displays a deprecation warning for the legacy builder and provides instructions to use BuildKit. The build process is shown as sending context to the daemon and pulling layers from the `python:3` image. The second screenshot shows the execution of `docker run hello-world-app`, which outputs "Hello World".

```
tenosho@tenosho-VirtualBox:~/devops-toolchain-milestone3/individuals/tenosho/docker$ docker build -t hello-world-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.584kB
Step 1/3 : FROM python:3
3: Pulling from library/python
61719a06ef52: Pulling fs layer
f32f49ce655a: Pulling fs layer
8d6d44b254da: Pulling fs layer
8a7504cd2818: Pulling fs layer
0a4465cc9f09: Pulling fs layer
b53089dca505: Pulling fs layer
c965dce520b3: Pulling fs layer
dbb6e7b34a8b: Download complete
61719a06ef52: Download complete
346dd1c2c849: Download complete
0a4465cc9f09: Download complete
8a7504cd2818: Download complete
c965dce520b3: Download complete
f32f49ce655a: Download complete
b53089dca505: Download complete
f32f49ce655a: Pull complete
8a7504cd2818: Pull complete
8d6d44b254da: Download complete
Successfully tagged hello-world-app:latest
tenosho@tenosho-VirtualBox:~/devops-toolchain-milestone3/individuals/tenosho/docker$ docker run hello-world-app
Hello World
tenosho@tenosho-VirtualBox:~/devops-toolchain-milestone3/individuals/tenosho/docker$
```

2. Git Practical Exercise

Objective

To demonstrate version control using Git.

Step 1: Navigate to the Repository

The Git repository provided by the group leader was accessed.

```
cd devops-toolchain-milestone3
```

Step 2: Check Repository Status

```
git status
```

This command displayed the current state of the repository.

Step 3: Edit index.html

The file was edited using Nano.

```
nano index.html
```

A line of content was updated.

Step 4: Stage the Changes

```
git add .
```

This command added modified files to the staging area.

Step 5: Commit the Changes

```
git commit -m "Updated index.html"
```

This command created a new commit containing the modifications.

Step 6: Push Changes to GitHub

```
git push
```

The changes were pushed to the remote GitHub repository.

Screenshot

```

temosho@temosho-VirtualBox:~/devops-toolchain-milestone3/individuals/temosho/git$ nano index.html
temosho@temosho-VirtualBox:~/devops-toolchain-milestone3/individuals/temosho/git$ git add .
temosho@temosho-VirtualBox:~/devops-toolchain-milestone3/individuals/temosho/git$ git commit -m "Updated index page"
[main 5426134] Updated index page
1 file changed, 1 insertion(+)
temosho@temosho-VirtualBox:~/devops-toolchain-milestone3/individuals/temosho/git$ git commit -m "Updated index page"
[main 5426134] Updated index page
1 file changed, 1 insertion(+)
create mode 100644 individuals/temosho/git/index.html
temosho@temosho-VirtualBox:~/devops-toolchain-milestone3/individuals/temosho/git$ git push
temosho@temosho-VirtualBox:~/devops-toolchain-milestone3/individuals/temosho/git$ git push
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 489 bytes | 489.00 KiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/Childish-kid75/devops-toolchain-milestone3.git
b7a23a7..5426134 main -> main

```

3. Chef Practical Exercise

Objective

To create and execute a Chef recipe that installs Nginx and manages a web service.

After encountering “cookbook not found” errors with chef-client, I used chef-apply – a valid Chef tool that runs a standalone recipe file. This satisfies the requirement to “create a recipe and run it locally”.

Steps performed:

1. Navigated to my individual Chef folder:

```
bash
```

```
cd ~/devops-toolchain-milestone3/individuals/temosho/chef
```

2. Created a recipe file nginx-recipe.rb with the following content:

```
ruby
```

```
package 'nginx' do
```

```
  action :install
```

```
end
```

```
service 'nginx' do
```

```
    action [:enable, :start]
```

```
end
```

```
file '/var/www/html/index.html' do
```

```
    content '<h1>Hello from temosho - Chef!</h1><p>Nginx installed with chef-  
apply</p>'
```

```
    mode '0644'
```

```
    action :create
```

```
end
```

3. Applied the recipe:

```
bash
```

```
sudo chef-apply nginx-recipe.rb
```

(If chef-apply was missing, I reinstalled Chef Workstation:

```
curl -L https://omnitruck.chef.io/install.sh | sudo bash)
```

4. Verified Nginx was running and serving the custom page:

```
bash
```

```
curl http://localhost
```

Output of curl http://localhost:

```
<h1>Hello from temosho - Chef!</h1><p>Nginx installed with chef-apply</p>
```

Screenshot

```

temosho@temosho-VirtualBox:~/devops-toolchain-milestone3/individuals/temosho/chef$ # 1. Go to your individual chef folder (adjust path if needed)
cd ~/devops-toolchain-milestone3/individuals/temosho/chef

# 2. Create the recipe file
cat > nginx-recipe.rb << 'EOF'
package 'nginx' do
  action :install
end

service 'nginx' do
  action [:enable, :start]
end

file '/var/www/html/index.html' do
  content '<h1>Hello from temosho - Chef!</h1><p>Nginx installed with chef-apply</p>'
  mode '0644'
  action :create
end
EOF

# 3. Run it with sudo
sudo chef-apply nginx-recipe.rb
Recipe: (chef-apply cookbook)::(chef-apply recipe)
 * apt_package[nginx] action install

   - install version 1.28.3-2ubuntu1.1 of package nginx
 * service[nginx] action enable (up to date)
 * service[nginx] action start (up to date)

Change mode from root to root

temosho@temosho-VirtualBox:~/devops-toolchain-milestone3/individuals/temosho/chef$ curl http://localhost
<h1>Hello from temosho - Chef!</h1><p>Nginx installed with chef-apply</p>temosho@temosho-VirtualBox:~/devops-toolchain-milestone3/individuals/temosho/chef$

```

Challenges Encountered

Several challenges were encountered during the practical, including VirtualBox startup issues, Docker installation configuration, and GitHub authentication errors. These issues were resolved through troubleshooting, updating configurations, and verifying service status using Linux commands.

Conclusion

The practical exercise successfully demonstrated the installation and use of Docker, Git, and Chef within an Ubuntu environment. Docker was used to containerize a Python application, Git was used for version control and collaboration, and Chef was used to automate server configuration. These tools form important components of a modern DevOps workflow and provided practical experience in software deployment, automation, and infrastructure management.